

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x05 实验报告

fork 的实现、并发与锁机制

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. fork 的实现

1.1. 系统调用

首先, 我们设置系统调用号为 `58`, 将用户态库中给出的系统调用对接到内核对应的函数当中，部分代码如下

```
#[inline(always)]
pub fn sys_fork() -> u16 {
    syscall!(Syscall::Fork) as u16
}

Syscall::Fork => {
    crate::proc::fork(context);
}
```

1.2. 进程管理

首先，我们实现最外层的 `fork`

```
pub fn fork(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let manager = get_process_manager();
        // FIXED: save_current as parent
        manager.save_current(context);
        // FIXED: fork to get child
        manager.fork();
        // FIXED: push to child & parent to ready queue
        // FIXED: switch to next process
        manager.switch_next(context);
    })
}
```

具体而言，就是保存当前上下文，然后调用 `ProcessManager::fork` 委托给下一层处理，然后再切换到下一进程的上下文，这里的将子进程放入就绪队列的操作在下层 `fork` 实现。

然后，实现 `ProcessManager::fork`

```rust
pub fn fork(&self) {
    // FIXED: get current process
    let current = self.current();
    // FIXED: fork to get child
    let child = current.fork();
    let child_pid = child.pid();
    // FIXED: add child to process list
    self.add_proc(child_pid, child.clone());
    self.push_ready(child_pid);
    // FOR DBG: maybe print the process ready queue?
    log::trace!(
        "Process {:?} successfully forked child {:?}",
        current.pid(),
        child_pid
    );
}
```

这里是将当前的 `fork` 任务委托到下一层进程自己的 `Process::fork` 执行，然后本层要做的就是把子进程放入就绪队列并加入进程管理表中。

接着实现 `Process::fork`

```rust
pub fn fork(self: &Arc<Self>) -> Arc<Self> {
    // FIXED: lock inner as write
    let mut inner = self.write();
    // FIXED: inner fork with parent weak ref
    let child_inner = inner.fork(Arc::downgrade(self));

    // FOR DBG: maybe print the child process info
    //          e.g. parent, name, pid, etc.
    log::trace!(
        "Process {}#{} forked a child process with pid: {}",
        inner.name(),
        self.pid,
        child_inner.parent().unwrap().pid()
    );
    // FIXED: make the arc of child
    let child = Arc::new(Self {
        pid: ProcessId::new(),
        inner: spin::RwLock::new(child_inner),
    });
    // FIXED: add child to current process's children list
    inner.children.push(child.clone());
    // FIXED: set fork ret value for parent with `context.set_rax`
    inner.context.set_rax(child.pid().0 as usize);
    // FIXED: mark the child as ready & return it
    child
}
```

这一层需要做的是，委托到 `ProcessInner::fork`，然后把当前获得的子进程的 `ProcessInner` 和其 `pid` 打包，并放入当前进程的子进程表中，并设置父进程的 `rax` 返回值寄存器为子进程的 `pid`。

然后实现的是 `ProcessInner::fork`

```
pub fn fork(&mut self, parent: Weak<Process>) -> ProcessInner {
    // FIXED: fork the process virtual memory struct
    // FIXED: calculate the real stack offset
    let stack_offset_count = self.children.len() as u64 + 1;
    let child_vm = self.vm().fork(stack_offset_count);
    // FIXED: clone the process data struct
    let mut child_context = self.context.clone();

    // FIXED: update `rsp` in interrupt stack frame
    // FIXED: set the return value 0 for child with `context.set_rax`

    // let stack_offset = stack_offset_count * STACK_MAX_SIZE;
    let parent_stack_base = self.vm().stack.range.start.start_address().as_u64();
    let child_stack_base = child_vm.stack.range.start.start_address().as_u64();
    let stack_offset = parent_stack_base - child_stack_base;

    child_context.as_mut().stack_frame.stack_pointer -= stack_offset;
    child_context.as_mut().regs.rbp -= stack_offset as usize;
    child_context.set_rax(0);
    // FIXED: construct the child process inner
    let child_data = self.proc_data.clone();
    // NOTE: return inner because there's no pid record in inner
    ProcessInner {
        name: self.name.clone(),
        parent: Some(parent),
        children: Vec::new(),
        ticks_passed: 0,
        status: ProgramStatus::Ready,
        context: child_context,
        exit_code: None,
        proc_data: child_data,
        proc_vm: Some(child_vm),
    }
}
```

这一层需要做的是，委托到内存管理器，建立对应的栈，然后本层做的是计算出对应的栈指针地址，这里需要 `+1` 是因为上层 `fork` 是先调用了当前的 `fork` 之后再把子进程插入子进程 `Vec` 中，此后需要修改栈指针，设置子进程的 `rax` 返回值寄存器的值为 `0`，然后构造出子进程的 PCB 并返回。

在实现的时候，本来使用 `stack_offset_count * STACK_MAX_SIZE` 计算偏移的，但是在测试的时候出现了地址的冲突，原因在于 `map_range` 内部会调整冲突的偏移，因此需要用父子进程的栈起始地址之差进行计算。

最后要实现的就是栈的建立和复制了，如下

```
pub fn fork(
    &self,
    mapper: MapperRef,
    alloc: FrameAllocatorRef,
    stack_offset_count: u64,
) -> Self {
    // FIXED: alloc & map new stack for child (see instructions)
    let cur_stack_base = self.range.start.start_address().as_u64();
    let mut new_stack_base = cur_stack_base - stack_offset_count *
STACK_MAX_SIZE;

    // log::info!("1. current: {}, new: {}", cur_stack_base, new_stack_base);
    // log::info!("usage: {} range: {}", self.usage, cur_stack_base);

    while elf::map_range(new_stack_base, self.usage, mapper, alloc).is_err() {
        trace!("Map thread stack to {:#x} failed.", new_stack_base);
        new_stack_base -= STACK_MAX_SIZE; // stack grow down
    }
    // log::info!("2. current: {}, new: {}", cur_stack_base, new_stack_base);
    // FIXED: copy the *entire stack* from parent to child
    self.clone_range(cur_stack_base, new_stack_base, self.usage);
    // FIXED: return the new stack
    Self {
        range: Page::range(
            Page::containing_address(VirtAddr::new(new_stack_base)),
            Page::containing_address(VirtAddr::new(new_stack_base)) + self.usage,
        ),
        usage: self.usage,
    }
}
```

具体而言，就是先计算出子进程的栈的栈底，然后建立页表映射，最后复制父进程的数据，把栈的 `range` 和 `usage` 信息打包后返回。

然后尝试运行 `fork_test` 进行测试，

```
[>] fork_test
I am the child process
I am the parent process
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  3 | #  2 | fork_test    |        4 | Running | 28672 B (7 Pages)
 #  2 | #  1 | sh           |    13201 | Ready | 28672 B (7 Pages)
 #  4 | #  3 | fork_test    |        3 | Ready | 28672 B (7 Pages)
 #  1 | #  0 | kernel       |    13216 | Ready | 2097152 B (512 Pages)
Queue  : [4, 2, 1]
CPUs   : [0: 3]
Waiting for child to exit...
child read value of M: 0xdeadbeef
child changed the value of M: 0x2333
Child exited with status 64
parent read value of M: 0x2333
Process [3] exited with code: 1056
```

可以看到，`fork_test` 正常运行并退出，可知 `fork` 的实现大致无误。

2. 进程的阻塞与唤醒

这个部分的实现比较简单，主要就是把 `wait_pid` 从轮询的方式改为阻塞，每次将一个需要等待的进程放入被等待进程的等待队列中然后阻塞（设置为 `blocked` 状态）。当那个被等待的进程被 `kill` 了之后，把所有等待它的进程唤醒，将状态改为 `ready` 并放入就绪队列。

实现了基于阻塞的 `wait_pid` 后进行测试，可以看到原本的 `sh` 被阻塞，当当前的 `sh` 退出后，又调度回到 `running` 态，可知这部分功能实现大致无误。

```
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] sh
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] stat
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  3 | #  2 | sh           |   14418 | Running | 28672 B (7 Pages)
 #  1 | #  0 | kernel       |   22969 | Ready | 2097152 B (512 Pages)
 #  2 | #  1 | sh           |    8542 | Blocked | 28672 B (7 Pages)
Queue  : [1]
CPUs   : [0: 3]
[>] exit
Goodbye!
Process [3] exited with code: 0
[>] stat
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  1 | #  0 | kernel       |  225334 | Ready | 2097152 B (512 Pages)
 #  2 | #  1 | sh           |   17514 | Running | 28672 B (7 Pages)
Queue  : [1]
CPUs   : [0: 2]
[>] exit
Goodbye!
[INFO] ysos_kernel - YatSenOS shutting down.
```

3. 并发与锁机制

3.1. 自旋锁

只需要使用 `compare_exchange` 和 `store` 这两个原子操作来维护 `AtomicBool` 变量即可。

```rust
pub fn acquire(&self) {
    // FIXED: acquire the lock, spin if the lock is not available
    while self
        .bolt
        .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
        .is_err()
    {
        spin_loop();
    }
}

pub fn release(&self) {
    // FIXED: release the lock
    self.bolt.store(false, Ordering::Release);
}
```

3.2. 信号量

对于信号量结构体的实现，如下

```
pub fn wait(&mut self, pid: ProcessId) -> SemaphoreResult {
    // FIXED: if the count is 0, then push pid into the wait queue
    //          return Block(pid)
    // FIXED: else decrease the count and return Ok
    if self.count == 0 {
        self.wait_queue.push_back(pid);
        SemaphoreResult::Block(pid)
    } else {
        self.count -= 1;
        SemaphoreResult::Ok
    }
}

pub fn signal(&mut self) -> SemaphoreResult {
    // FIXED: if the wait queue is not empty
    //          pop a process from the wait queue
    //          return WakeUp(pid)
    // FIXED: else increase the count and return Ok
    if let Some(pid) = self.wait_queue.pop_front() {
        SemaphoreResult::WakeUp(pid)
    } else {
        self.count += 1;
        SemaphoreResult::Ok
    }
}
```

具体而言，`wait` 就是尝试获取锁，如果锁还可以使用则 `-1` 并返回 `Ok`，否则加入该锁的等待队列并返回 `Block`；而 `signal` 则是释放锁，如果等待队列非空则唤醒其中的一个进程，否则让信号量 `+1`。

然后要实现的是保存所有信号量的 `Key-Value` 存储，只需要使用一个 `HashMap` 即可，其他操作使用 `HashMap` 的成员函数即可。这里不再赘述，具体请见 `proc/sync.rs`。

最后要实现的是系统调用中对信号量的 `new`，`remove`，`signal` 以及 `wait` 的处理（包括相关的委托到 `sync.rs` 中的函数，根据返回值来进行对应的处理等），具体实现请见 `syscall/service.rs`。

3.3. 测试任务

3.3.1. 多线程计数器

测试的代码请见 `app/counter/src/main.rs`，对于自旋锁，使用 `lib::sync::SpinLock::new()` 声明一个锁，然后每次自增 `COUNTER_SPIN` 的时候，需要先 `SPIN_LOCK.acquire()` 获取锁，然后修改之后需要 `SPIN_LOCK.release()` 释放锁；对于信

号量，先使用 `sys_new_sem(2333, 1)` 声明一个 `key` 为 `2333`，`value` 为 `1` 的信号量，然后每次自增 `COUNTER_SEMAPHORE` 前需要 `sys_sem_wait(2333)` 获取锁，修改之后需要 `sys_sem_signal(2333)` 释放锁。

运行该程序，两个 `COUNTER` 均为 `800`，说明自旋锁和信号量的实现大致无误。

```
[>] counter
Start SpinLock test...
Start Semaphore test...
SpinLock COUNTER result: 800
Semaphore COUNTER result: 800
Process [3] exited with code: 0
```

3.3.2. 消息队列

代码参见 `app/mq/src/main.rs`，目前的代码可以在生产者生产信息以及消费者取出信息的时候输出日志。设置缓冲区的大小为 `1` 时，可以观察到如下输出

```
Producer 2 pushed message: 2000 | Count: 1
Consumer 2 popped message: 2000 | Count: 0
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  3 | #  2 | mq           |      44 | Running | 32768 B (8 Pages)
 # 12 | #  3 | mq           |      11 | Blocked | 32768 B (8 Pages)
 # 17 | #  3 | mq           |       2 | Blocked | 32768 B (8 Pages)
 # 16 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 # 13 | #  3 | mq           |       2 | Blocked | 32768 B (8 Pages)
 #  9 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 # 11 | #  3 | mq           |       2 | Blocked | 32768 B (8 Pages)
 #  2 | #  1 | sh           |    6615 | Blocked | 28672 B (7 Pages)
 #  8 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 # 18 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 # 10 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 # 19 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 #  1 | #  0 | kernel       |    6667 | Ready | 2097152 B (512 Pages)
 #  6 | #  3 | mq           |       2 | Ready | 32768 B (8 Pages)
 #  4 | #  3 | mq           |       4 | Blocked | 32768 B (8 Pages)
 # 15 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 #  7 | #  3 | mq           |       1 | Blocked | 32768 B (8 Pages)
 #  5 | #  3 | mq           |       4 | Blocked | 32768 B (8 Pages)
 # 14 | #  3 | mq           |       5 | Blocked | 32768 B (8 Pages)
Queue  : [6, 1]
CPUs   : [0: 3]
Producer 1 pushed message: 1000 | Count: 1
Consumer 0 popped message: 1000 | Count: 0
Producer 3 pushed message: 3000 | Count: 1
Consumer 3 popped message: 3000 | Count: 0
```

可以看到，`main.rs` 创建了 `16` 个子进程分别作为 `producer` 或者 `consumer`，然后由于此时缓冲区大小为 `1`，可以观察到生产者消费者轮流从缓冲区中加入消息，取出消息，同时插入的消息和取出的消息内容一致，可知实现大致无误。

当我们把缓冲区大小改为 `4` 的时候，可以观察到如下输出

```
Producer 4 pushed message: 4009 | Count: 4
Consumer 2 popped message: 2009 | Count: 3
Consumer 3 popped message: 5008 | Count: 2
Consumer 4 popped message: 6009 | Count: 1
Consumer 5 popped message: 4009 | Count: 0
Producer 7 pushed message: 7009 | Count: 1
Producer 5 pushed message: 5009 | Count: 2
Consumer 7 popped message: 7009 | Count: 1
Consumer 6 popped message: 5009 | Count: 0
All finished. Remaining messages: 0
Process [3] exited with code: 0
```

队列的元素计数是否没有超过容量，且可以达到上界 `4`，生产者和消费者能够交替工作，且可以观察到多个生产者或消费者同时进行同种操作的情况，同时取出的信息也和放入的信息对应，到最后消息队列中的元素个数恰好为 `0` 。

尝试把日志输出级别改为 `Debug`，然后再次运行可以观察到如下输出，`Process 4 ~ 11` 是 `consumer`，`Process 12 ~ 19` 是 `producer`，当消息队列为空时，所有的 `consumer` 都被阻塞

```
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 6 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 8 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 7 is blocked
Process 8 is created as consumer
Process 9 is created as consumer
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 9 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 10 is blocked
Process 10 is created as consumer
Process 11 is created as consumer
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 11 is blocked
Process 12 is created as producer
Producer 0 pushed message: 0 | Count: 1
Process 13 is created as producer
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 4 is woken up
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 4 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 12 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 14 is blocked
Producer 1 pushed message: 1000 | Count: 2
```

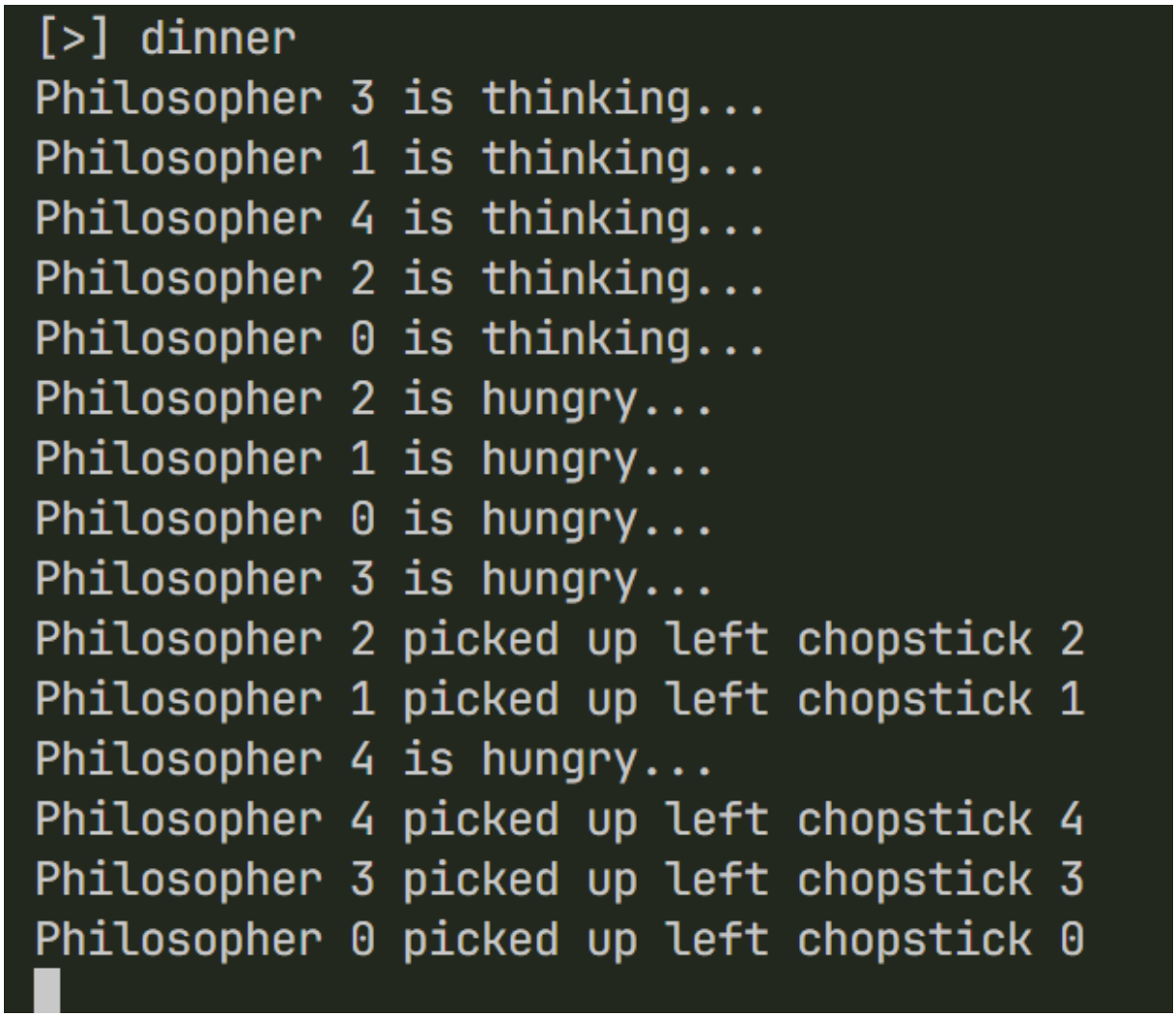
我们可以修改成先 `fork` 出来 `producer`，就可以观察到当消息队列满了的时候，所有的 `producer` 都被阻塞，

```
Process 4 is created as producer
Producer 0 pushed message: 0 | Count: 1
Producer 0 pushed message: 1 | Count: 2
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 5 is blocked
Process 5 is created as producer
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 5 is woken up
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 4 is blocked
Process 6 is created as producer
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 6 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 7 is blocked
Process 7 is created as producer
Process 8 is created as producer
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 8 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 9 is blocked
Producer 1 pushed message: 1000 | Count: 3
Process 9 is created as producer
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 4 is woken up
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 10 is blocked
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 5 is blocked
Process 10 is created as producer
Process 11 is created as producer
Producer 0 pushed message: 2 | Count: 4
[DEBUG] ysos_kernel::interrupt::syscall::service - Process 4 is blocked
Process 12 is created as consumer
```

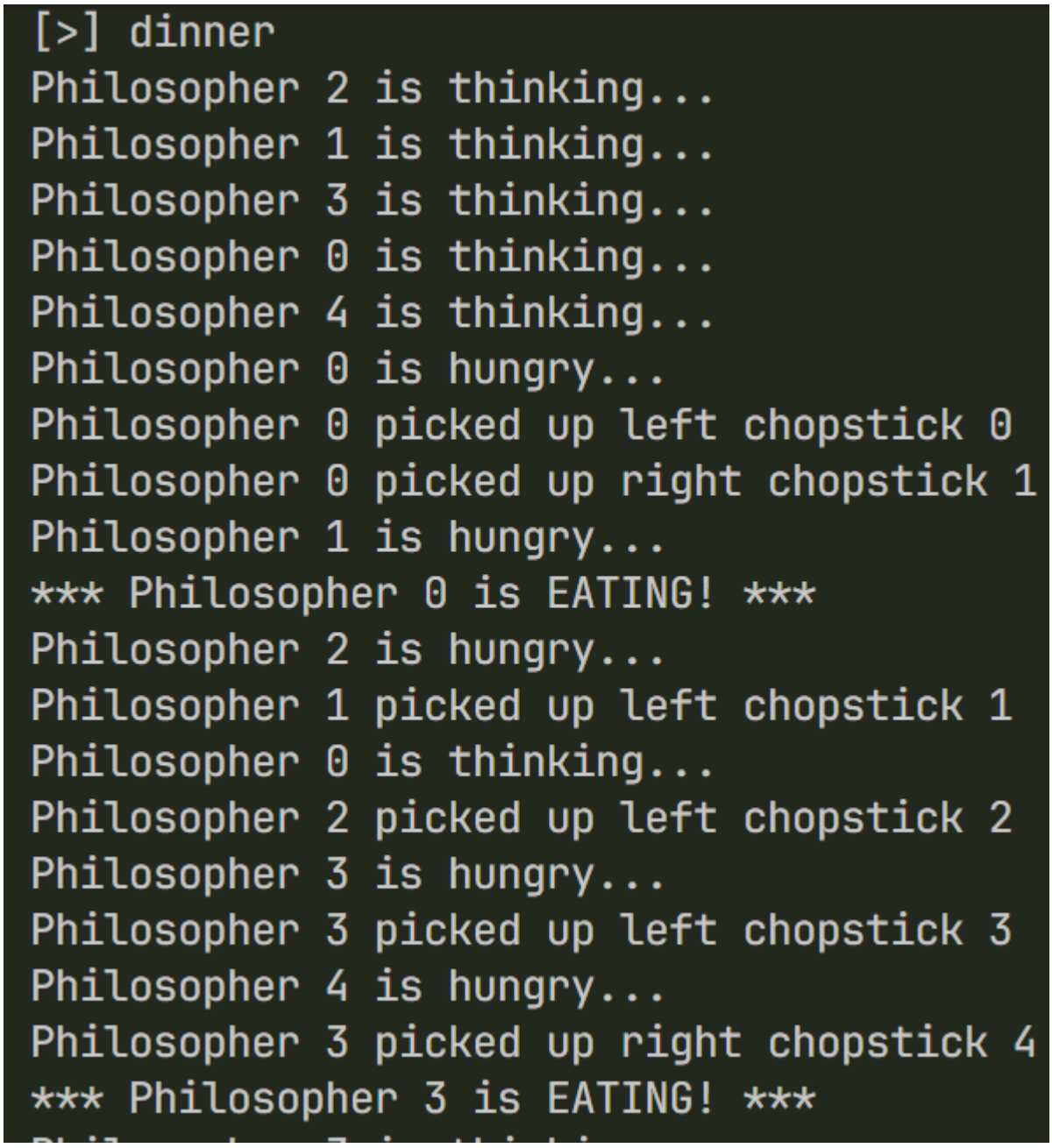
从而我们证明了队列的正常工作。

3.3.3. 哲学家的晚饭

代码参见 `app/dinner/src/main.rs`，我们的设计是让每个人先拿起左手的筷子，然后再拿起右手的筷子，如果我们把哲学家的思考的时间都调得比较小，然后拿起左筷子之后都会等一段时间再拿右筷子，所有哲学家都拿起了一只筷子，而拿不起第二只筷子，就会造成死锁的情况，如下



如果我们适当的调整思考时间和拿起筷子的间隔时间, 就能构造出哲学家能吃到饭的情况, 如下



如果我们改变其中 `0` 号哲学家的策略为先拿起右筷子，后拿起左筷子，就会由于别人先拿走了它的右筷子，在右筷子可用时左筷子又需要等待，而造成的饥饿现象，如下，其他四个哲学家都吃到了至少一次饭，才轮到 `0` 号哲学家吃饭。

```
Philosopher 4 picked up right chopstick 0
*** Philosopher 4 is EATING! ***
Philosopher 4 is thinking...
Philosopher 3 picked up right chopstick 4
*** Philosopher 3 is EATING! ***
Philosopher 4 is hungry...
Philosopher 3 is thinking...
Philosopher 4 picked up left chopstick 4
Philosopher 2 picked up right chopstick 3
*** Philosopher 2 is EATING! ***
Philosopher 3 is hungry...
Philosopher 4 picked up right chopstick 0
*** Philosopher 4 is EATING! ***
Philosopher 2 is thinking...
Philosopher 3 picked up left chopstick 3
Philosopher 1 picked up right chopstick 2
*** Philosopher 1 is EATING! ***
Philosopher 3 picked up right chopstick 4
Philosopher 2 is hungry...
Philosopher 4 is thinking...
*** Philosopher 3 is EATING! ***
Philosopher 1 is thinking...
Philosopher 4 is hungry...
Philosopher 0 picked up right chopstick 1
Philosopher 2 picked up left chopstick 2
Philosopher 1 is hungry...
Philosopher 3 is thinking...
Philosopher 2 picked up right chopstick 3
*** Philosopher 2 is EATING! ***
Philosopher 0 picked up left chopstick 0
*** Philosopher 0 is EATING! ***
Philosopher 4 picked up left chopstick 4
```

要解决上述出现的问题，我们可以参考实验文档加入一个 `Waiter` 信号量，信号量的个数为 `4`，这样就可以保证至多只有四个哲学家同时就餐，至少有一个哲学家可以获得两个筷子来吃饭，不会造成死锁的问题。我们在构造死锁的情况下加上这个 `Waiter` 信号量之后，可以观察到所有哲学家都正常用餐直到结束。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
*** Philosopher 2 is EATING! ***
Philosopher 0 is hungry...
Philosopher 1 picked up right chopstick 2
Philosopher 0 picked up left chopstick 0
*** Philosopher 1 is EATING! ***
Philosopher 0 picked up right chopstick 1
*** Philosopher 0 is EATING! ***
Philosopher 4 is hungry...
Philosopher 4 picked up left chopstick 4
Philosopher 4 picked up right chopstick 0
*** Philosopher 4 is EATING! ***
Dinner is over!
Process [38] exited with code: 0
```

4. 思考题

4.1. 在 Lab 2 中设计输入缓冲区时，如果不使用无锁队列实现，而选择使用 `Mutex` 对一个同步队列进行保护，在编写相关函数时需要注意什么问题？考虑在进行 `pop` 操作过程中遇到串口输入中断的情形，尝试描述遇到问题的场景，并提出解决方案。

需要注意会不会出现死锁，比如当 `pop` 操作进行时，此时内核取得了缓冲区得读写锁，此时遇到串口输出中断，中断处理程序抢占 `CPU` 自旋等待读写锁写入缓冲区，但此时的锁在内核手上无法释放，就造成了死锁。解决的方法时，在内核进行 `pop` 操作时，需要先关中断，防止其他中断自旋等待该锁造成死锁。

4.2. 在进行 `fork` 的复制内存的过程中，系统的当前页表、进程页表、子进程页表、内核页表等之间的关系是怎样的？在进行内存复制时，需要注意哪些问题？

在一般的 OS 的 `fork` 复制内存过程中，系统的当前页表指向父进程的页表，子进程页表采用 Cow 写时复制机制，内核页表共享。本实验中的简化实现是子进程和父进程共享一张页表，然后子进程设置自己独立的栈和寄存器。需要注意的是在实验文档中设计的这种栈布局在复杂情况下可能会造成栈复用，导致 `map_range` 失败，从而会带来偏移量的调整，因此在计算栈的偏移的时候需要以最终映射成功的内存地址之差来计算偏移，否则会造成内存复制时的冲突。

4.3. 为什么在实验的实现中，`fork` 系统调用必须在任何 `Rust` 内存分配（堆内存分配）之前进行？如果在堆内存分配之后进行 `fork`，会有什么问题？

因为我们的实现中，没有实现 Cow 写时复制机制，父子进程的堆是共享的，`fork` 只会给子进程建立自己的栈，因此 `fork` 系统调用需要在任何的堆内存分配之前进行，来保证子进程退出后 drop 了的堆变量是子进程自己的，而不是和父进程共享的。如果先进行了堆内存分配后 `fork` 则可能会带来 double-free 或者 use-after-free 的问题。

4.4. 进行原子操作时候的 `Ordering` 参数是什么？此处 Rust 声明的内容与 C++20 规范 中的一致，尝试搜索并简单了解相关内容，简单介绍该枚举的每个值对应于什么含义。

`Ordering` 参数对应的当前进程的指令顺序的排序严格程度。`Relaxed` 对应的是只保证当前操作是原子的，但是并不保证在其他进程的观测中中这些原子操作的顺序不变；`Acquire` 一般与 `Load` 结合使用，对应的是保证在其他线程的观测中，可以确定在这个 `Load` 操作之后的读写操作不会指令重排到其之前；`Release` 一般与 `Store` 结合使用，对应的是保证在其他线程的观测中，可以确定在这个 `Store` 指令之前的读写操作不会被重排到其之后，可以结合如下代码理解（参考自知乎用户 newbmiao 的回答）

```
let a = thread::spawn(|| {
    unsafe { DATA = 42 };
    FLAG.store(true, Ordering::Release);
    // 不允许有读写重排到 store 之后
    // 即后面的 FLAG 变为 true 之前一定已经写入了 42
});
let b = thread::spawn(|| {
    // 不允许有读写重排到 load 之前
    // 即后面的 DATA 读取到的一定是 42
    while !FLAG.load(Ordering::Acquire) {}
    assert_eq!(unsafe { DATA }, 42);
});

a.join().unwrap();
b.join().unwrap();
```

而 `AcqRel` 是上述两者的结合，一般用在 `fetch_add` 等原子操作上；最严格的 `SeqCst` 是在保证上述重排限制的前提下，另外保证所有线程都可以按相同的顺序看到所有顺序一致的操作（即前者可以保证的是单一原子变量各个原子操作之间的顺序，后者可以保证全局所有原子变量的操作之间的顺序）。如下方的代码，如果不使用 `SeqCst`，可能会导致两个 `Thread...` 都输出的情况，即使使用 `Acquire + Release` 也没用，因为这个只保证线程内部的 `Store` 一定在 `Load` 之前，但不能保证两个线程判断时看到的 `X, Y` 的修改顺序一致，除此以外，还要求所有 `SeqCst` 操作在所有线程看来有一个相同的全序（即不可能出现两个都能打印出来的情况）

```
let t1 = thread::spawn(|| {
    X.store(true, Ordering::SeqCst);
    if !Y.load(Ordering::SeqCst) {
        println!("Thread 1: X was set before Y was observed false.");
    }
});

let t2 = thread::spawn(|| {
    Y.store(true, Ordering::SeqCst);
    if !X.load(Ordering::SeqCst) {
        println!("Thread 2: Y was set before X was observed false.");
    }
});

t1.join().unwrap();
t2.join().unwrap();
```

4.5. 在实现 `SpinLock` 的时候，为什么需要实现 `Sync` trait？类似的 `Send` trait 又是什么含义?

`Sync` 的含义是如果一个类型 `T` 实现了 `Sync` trait，就表示 `&T` 不可变引用可以在多个线程之间安全地共享，即多个线程可以同时通过不可变引用来访问同一个值而不会引发数据竞争，如果不实现这个 trait 会通过不了编译。而 `Send` 指的是如果一个类型 `T` 实现了 `Send` trait，就表示它的所有权可以在线程之间安全地转移，即可以把一个 `T` 类型的值从一个线程 `move` 到另一个线程。即前者是可以共享使用这个锁，而后者是可以传递这个锁的所有权。

4.6. `core::hint::spin_loop` 使用的 `pause` 指令和 Lab 4 中的 `x86_64::instructions::hlt` 指令有什么区别？这里为什么不能使用 `hlt` 指令?

`hlt` 指令指的是 `Halt`，这是一个特权指令，会直接让 CPU 休眠直到一个外部中断来临；而 `pause` 指令是一个非特权指令，会让 CPU 会优化执行，提供较少的资源给当前线程，减少自旋等待对 CPU 的消耗。这里由于 `counter` 是一个用户态程序，不能使用 `hlt` 这种特权指

令；其次 `hlt` 需要外部中断才会唤醒，如果在关中断等特殊的时候使用可能会造成永久睡眠，而 `pause` 会一直自旋检查锁的释放，在实现自旋锁的时候后者的表现更加符合预期。

4.7. 如果将自旋锁替换为“可睡眠锁”，在内核态和用户态分别会带来哪些收益与风险？(可选思考题)

替换成“可睡眠锁”，从用户态来看，可以提升 CPU 的利用率，睡眠期间会让出 CPU 给其他进程使用，而自旋锁会自旋等待锁，持续占用 CPU；但是“可睡眠锁”也意味着要进行上下文的切换，如果各进程之间交互的频率非常密集，大量的上下文切换开销会大大降低性能。

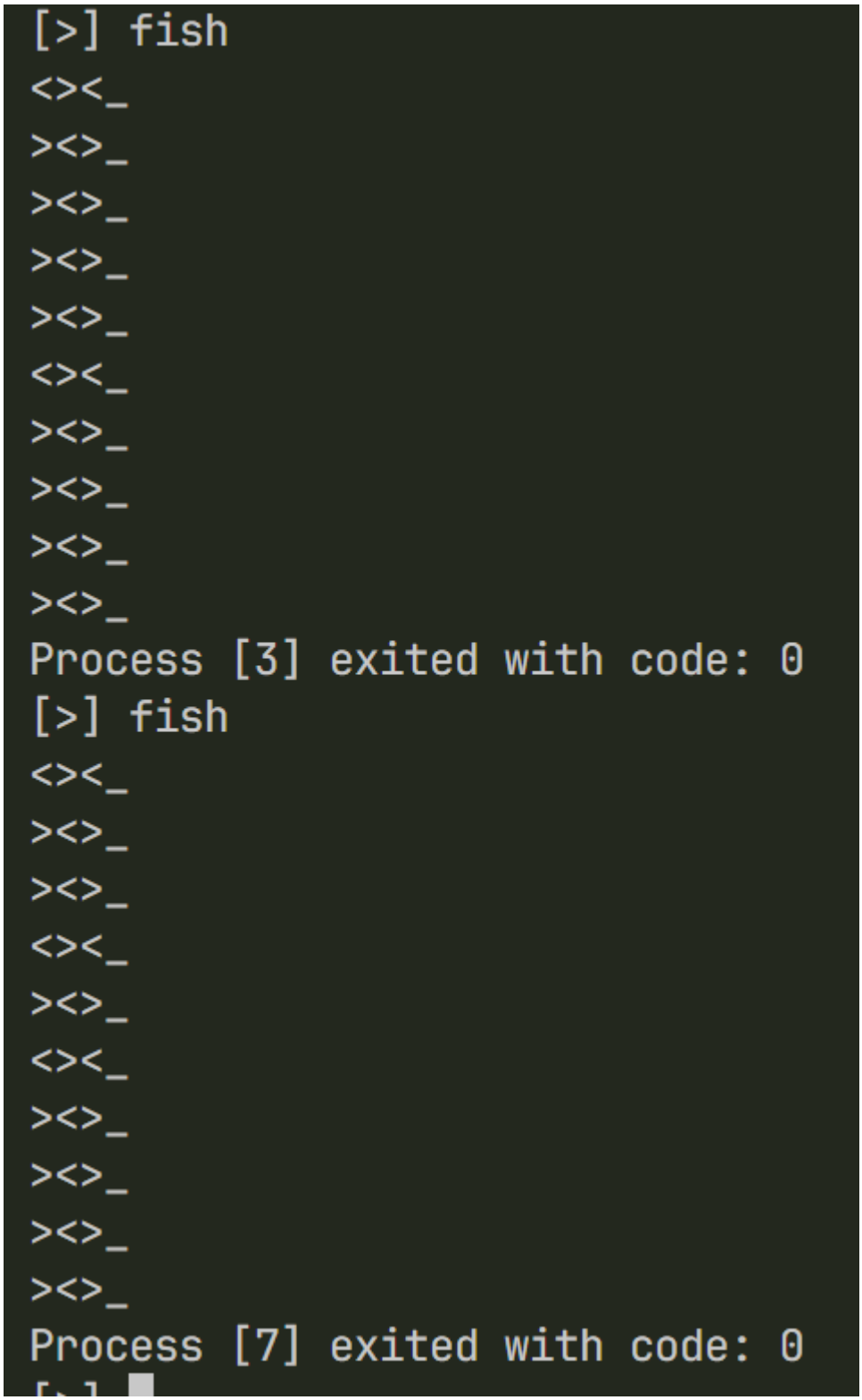
从内核态而言，等待 `I/O` 或者各种外部中断时，需要等待的时间确实会比较长，这些情况下为了节省 CPU 资源一般都会采用“可睡眠锁”，而且在内核进行进程调度的时候，可以用这种方式让一些低优先级的进程睡眠，让出 CPU 给高优先级任务；风险是需要大量的上下文切换，同时如果在关中断或者抢占时的时候使用了“可睡眠锁”，可能会导致死锁的问题。

5. 加分项

5.1. 实现 `fish` 用户程序

我们的实现方法是：构建一个状态机，每个状态分别表示当前打印到 `fish` 的第几个字符（`STATE`），以及当前打印的 `fish` 头朝左还是右（`DIR`），维护一个自旋锁 `SPIN_LOCK`，三个子进程取得锁后检查当前状态是否可以输出，如果不行则释放锁后使用 `core::hint::spin_loop` 等待，给其他子进程取得锁来进行输出，输出之后释放锁。

具体的代码参见 `app/fish/src/main.rs`，我们尝试运行这个代码，可以看到程序的输出符合预期，鱼的朝向是随机的，



5.2. 尝试和前文不同的其他方法解决哲学家就餐问题, 并验证你的方法能够正确解决它，简要介绍你的方法，并给出程序代码和测试结果。

一种和前文不同的方案是：考虑死锁的原因是所有哲学家都拿起了左筷子，而拿不到右筷子, 因此我们可以让所有编号为奇数的哲学家先拿起左筷子, 而编号为偶数的哲学家先拿起右边

筷子，可以证明这样至少有一个哲学家能吃上饭，从而不会造成死锁的情况，代码如下（其实就是加了个编号的判断）

```
if id % 2 == 0 {
    sys_sem_wait(CHOPSTICKS[right]);
    println!("Philosopher {} picked up right chopstick {}", id, right);

    sleep(0x100);

    sys_sem_wait(CHOPSTICKS[left]);
    println!("Philosopher {} picked up left chopstick {}", id, left);
} else {
    sys_sem_wait(CHOPSTICKS[left]);
    println!("Philosopher {} picked up left chopstick {}", id, left);

    sleep(0x10);

    sys_sem_wait(CHOPSTICKS[right]);
    println!("Philosopher {} picked up right chopstick {}", id, right);
}
```

然后保持之前构造死锁的设定不变，注释掉 `WAITER` 相关的信号量，进行测试，可以看到哲学家们可以正常吃饭。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
*** Philosopher 2 is EATING! ***
Philosopher 0 picked up right chopstick 1
Philosopher 1 is hungry...
Philosopher 3 picked up left chopstick 3
Philosopher 0 picked up left chopstick 0
*** Philosopher 0 is EATING! ***
Philosopher 3 picked up right chopstick 4
Philosopher 1 picked up left chopstick 1
*** Philosopher 3 is EATING! ***
Philosopher 1 picked up right chopstick 2
*** Philosopher 1 is EATING! ***
Dinner is over!
Process [3] exited with code: 0
[>]
```

5.3. 尝试使用符合 Rust 做法的方式处理互斥锁，使用 RAII 的方式来保证锁的释放

我们的目标是：每次要获得锁的时候创建一个 `SpinLockGuard` 类型变量（生命周期在其生命的代码块内），在离开该代码块时，这个变量自动调用 `drop` 时释放锁。我们可以对每个需要锁的数据提供专门的锁，首先我们在锁的定义中加入 `data`

```
pub struct SpinLock<T> {
    bolt: AtomicBool,
    data: core::cell::UnsafeCell<T>,
}
```

Rust 为了避免数据竞争，对于每个变量，要么只能有若干个共享引用，要么只能有一个可变引用，而我们预期的行为是每个子进程都可以执行 `.lock(&self)` 来获取可变引用，因此这里需要使用 `core::cell::UnsafeCell<T>`，它允许内部的 `data` 可以在共享引用的时候进行修改。它提供了 `get(&self) -> *mut T` 方法，可以安全地获得内部的可变指针，从而有办法获得内部的可变引用。如下

```
impl<T> DerefMut for SpinLockGuard<'_, T> {
    #[inline(always)]
    fn deref_mut(&mut self) -> &mut Self::Target {
        unsafe { &mut *self.lock.data.get() }
    }
}
```

上面的 `SpinLockGuard` 定义如下，其内部引用的锁生命周期不小于该变量，保证 `drop` 的时候可以正常释放锁，同时声明为 `'a` 而不是 `'static` 让我们可以在代码块内部创建一个带锁的变量。

```
pub struct SpinLockGuard<'a, T> {
    lock: &'a SpinLock<T>,
}
```

然后实现对应的 `drop`，在释放该变量的时候释放锁

```
impl<T> Drop for SpinLockGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.bolt.store(false, Ordering::Release);
    }
}
```

从而，我们获取一个数据的锁的时候，需要设置锁标记为 `true`，同时返回这个 `SpinLockGuard` 类型变量，使得用户可以获得内部数据的引用，同时在离开代码块的时候也可也自动释放锁。`.lock(&self)` 的代码如下，

```rust
pub fn lock(&self) -> SpinLockGuard<'_, T> {
    // FIXED: acquire the lock, spin if the lock is not available
    while self
        .bolt
        .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
        .is_err()
    {
        spin_loop();
    }
    SpinLockGuard { lock: self }
}
```

其余部分代码请见 `lib/src/sync.rs` 。

声明一个带锁的变量以及在一个代码块中锁住这个变量并修改，可以参考如下的代码

```rust
// init val as SpinLock type
let val: lib::sync::SpinLock<isize> = lib::sync::SpinLock::new(0);
// for example
{
    let mut val_guard = val.lock();
    *val_guard += 1;
}
// val has been released
```

由于这个锁是在离开代码块的时候释放的，因此我们为了避免卡顿，需要尽早离开代码块释放锁（或者也可以显式调用 `drop` ）。在实现 `fish` 的时候，由于之前没有考虑到这个问题，在持有锁的时候进行 `spin_loop()` ，导致程序卡顿，在真正使用锁的地方加上花括号包裹就恢复正常了。

给 `counter` 和 `fish` 使用这个新的锁进行测试，最终的表现与原本的预期一致，可知上面的实现无误。

但是上面的实现仍然不够安全，存在一个函数叫做 `mem::forget()` ，可以让一个变量在退出代码块的时候不调用 `drop` 函数，从而造成锁无法释放，带来一个我们无法处理的死锁。查询 `spin` crate 中的实现，其中提供了一个 `unsafe` 的 `unlock_force()` 方法给用户，来强制释放锁。这里由于没有这么复杂的情况，目前没有实现这个方法。